```
import heapq

# Creating Huffman tree node
class node:
    def __init__(self,freq,symbol,left=None,right=None):
        self.freq=freq # frequency of symbol
        self.symbol=symbol # symbol name (character)
        self.left=left # node left of current node
        self.right=right # node right of current node
        self.huff= '' # # tree direction (0/1)

    def __lt__(self,nxt): # Check if curr frequency less than next nodes freq
        return self.freq<nxt.freq

def printnodes(node,val=''):
    newval=val+str(node.huff)
    # if node is not an edge node then traverse inside it
    if node.left:
        printnodes(node.left,newval)
    if node.right:
        printnodes(node.right,newval)

    # if node is edge node then display its huffman code
    if not node.left and not node.right:
        print("{} -> {}".format(node.symbol,newval))

if __name__=="__main__":
    chars = ['a', 'b', 'c', 'd', 'e', 'f']
    freq = [ 5, 9, 12, 13, 16, 45]
    nodes=[]
```

```
for i in range(len(chars)): # converting characters and frequencies into huffman tree nodes
    heapq.heappush(nodes, node(freq[i],chars[i]))

while len(nodes)>1:
    left=heapq.heappop(nodes)
    right=heapq.heappop(nodes)

    left.huff = 0
    right.huff = 1
    # Combining the 2 smallest nodes to create new node as their parent
    newnode = node(left.freq + right.freq , left.symbol + right.symbol , left , right)
    # node(freq,symbol,left,right)
    heapq.heappush(nodes, newnode)

printnodes(nodes[0]) # Passing root of Huffman Tree
```